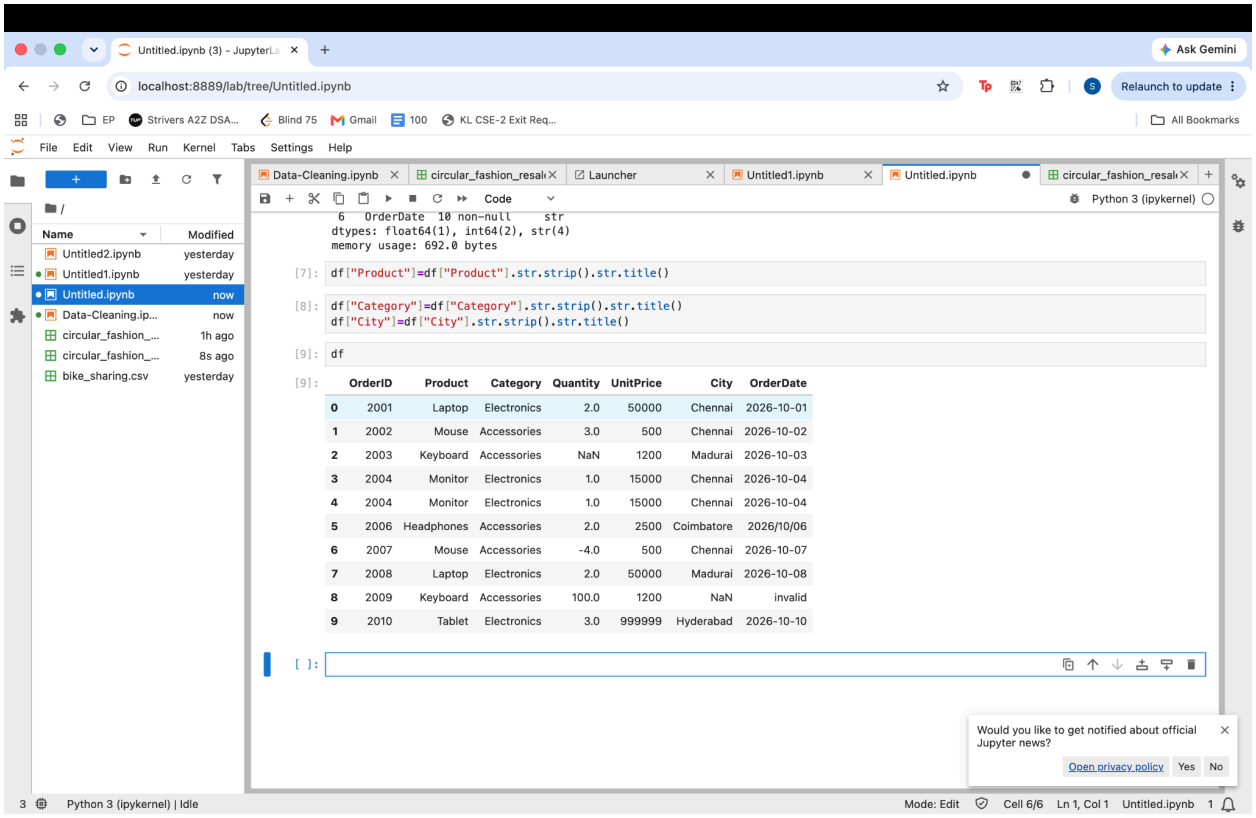
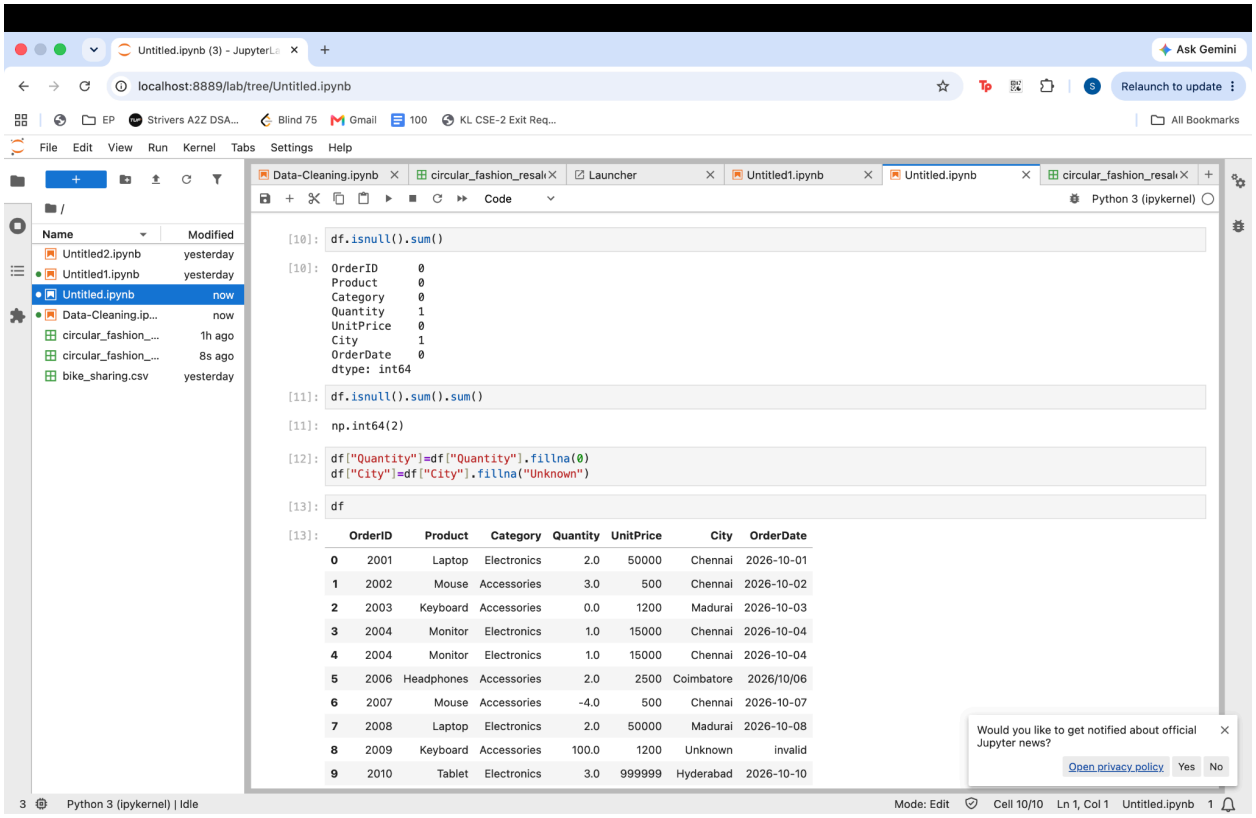


Data Formatting:

Changing the format of data into a consistent and appropriate format improves the accuracy and efficiency of business operations. For example, converting order dates into a proper datetime format allows the business to analyze daily, monthly, and yearly sales trends, while converting quantity and price columns into numeric formats enables accurate calculations of revenue, costs, and profits. Standardizing text formats, such as product names and city names, prevents the same product or location from being treated as different entries due to capitalization or extra spaces. These changes make filtering, sorting, grouping, and reporting more reliable, reduce calculation errors, and help management identify sales trends and make better business decisions.



Missing data identified:

Findings: `Quantity` has 1 missing value, `City` has 1, and `OrderDate` contains an invalid date string (`"invalid"`) that is not currently treated as missing.

Possible causes: Incomplete order entry, data import errors, or missing customer/order information.

Impact: Missing quantities can lead to underestimated revenue, missing cities can distort geographical analysis, and invalid dates can affect time-based sales reports.

```
[14]: df[df.duplicated(subset="OrderID", keep=False)]
```

[14]:

	OrderID	Product	Category	Quantity	UnitPrice	City	OrderDate
3	2004	Monitor	Electronics	1.0	15000	Chennai	2026-10-04
4	2004	Monitor	Electronics	1.0	15000	Chennai	2026-10-04

Found Repeated Product values:

Findings: Order ID `2004` appears twice.

Possible causes: Duplicate data entry, repeated imports, or multiple records belonging to the same order.

Impact: If the record is an accidental duplicate, revenue and order counts may be overstated. Verify the order details before deleting it.

```
highest = df.loc[df["UnitPrice"].idxmax(), ["Product", "UnitPrice"]]
print(highest)
```

```
Product      Tablet
UnitPrice    999999
Name: 9, dtype: object
```

Found Highest priced product:
Findings: The highest-priced product is Tablet, priced at ₹999,999.

Possible causes: A data-entry error, an incorrect currency/unit, or a genuinely expensive product.

Impact: An unusually high price can distort average prices, product comparisons, and pricing decisions. Verify it before changing the value.

```
df["Quantity"] = df["Quantity"].mask(df["Quantity"] < 0, np.nan)
df["Revenue"] = df["Quantity"] * df["UnitPrice"]

print(df.groupby("City")["Revenue"].sum().sort_values(ascending=False))
```

```
City
Hyderabad     2999997.0
Chennai        131500.0
Unknown        120000.0
Madurai        100000.0
Coimbatore       5000.0
Name: Revenue, dtype: float64
```

Grouped by City and got revenue sum:

Findings: Revenue is calculated city-wise. Missing cities are categorized as `Unknown`, while negative quantities and missing quantities produce missing revenue values.

Possible causes: Invalid order quantities, incomplete records, returns, or incorrect data entry.

Impact: Incorrect quantities can overstate or understate revenue. Missing city information can hide geographical sales patterns and lead to poor decisions about marketing, inventory, and regional performance.